Stack Problem Bank (Java)

Legend
★ = Core Placement Problem (Mandatory)
Covers internal implementation details alongside application-level problems.
These problems should be completed by every student before moving to the Collections Utility Class.

Section 1: Stack Basics and Creation

★ **Create a Stack using java.util.Stack**

★ **Create a Stack using ArrayDeque (Recommended Approach)**

Find Size of a Stack using size()

Check if a Stack is Empty using isEmpty()

Section 2: Stack Internals — Vector-Based Stack vs Deque-as-Stack (Core Concept)

★ **Internal Structure of java.util.Stack (Extends Vector, Synchronized Array)**

★ **Internal Structure of ArrayDeque as a Stack (Resizable Circular Array)**

★ **Why ArrayDeque is Recommended over java.util.Stack (Performance & Legacy Issues)**

Synchronization Overhead in Vector-Based Stack

★ **Time Complexity Comparison: push()/pop() — Stack vs ArrayDeque**

★ **Demonstrate LIFO Behavior Internally**

Section 3: Stack Operations

★ **Push Elements onto a Stack**

★ **Pop Elements from a Stack**

★ **Peek at the Top Element**

Search for an Element's Position using search()

Iterate through a Stack from Top to Bottom

Section 4: Stack-Based Logical Problems

★ **Reverse a String using Stack**

★ **Check for Balanced Parentheses using Stack**

★ **Check if a Sequence is a Palindrome using Stack**

★ **Sort a Stack using Another Stack**

Remove Adjacent Duplicates using Stack

★ **Find the Next Greater Element using Stack**

★ **Min Stack Implementation (getMin() in O(1))**

Section 5: Expression Evaluation Problems

★ **Infix to Postfix Conversion using Stack**

Infix to Prefix Conversion using Stack

★ **Evaluate Postfix Expression using Stack**

Evaluate Prefix Expression using Stack

Evaluate Infix Expression Directly using Two Stacks

Section 6: Real World Stack Applications

★ **Undo Feature Simulation using Stack**

★ **Browser Back Button Navigation using Stack**

Call Stack Simulation (Function Call Tracking)

★ **Bracket Matching Validator for Code Editors**

Backtracking Maze Solver using Stack